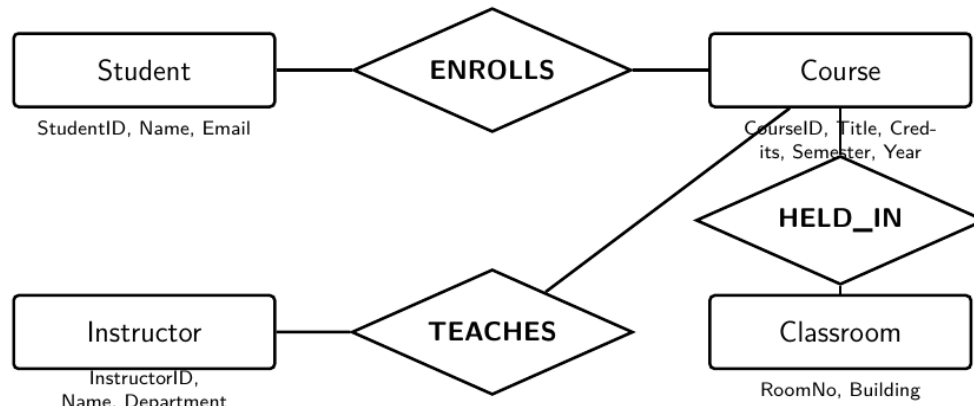


ER Forensics: Answers

Question 1: University Course Registration



As given in the assignment (page 3).

As given

- Entities: Student (StudentID, Name, Email), Course (CourseID, Title, Credits, Semester, Year), Instructor (InstructorID, Name, Department), Classroom (RoomNo, Building).
- Relationships: Student ENROLLS in Course, Instructor TEACHES Course, Course is HELD_IN Classroom.
- Semester and Year are shown as attributes of Course itself, not of any separate offering or section.

Issue 1: Semester and Year sit on Course

The flaw. A course and a specific offering of that course are two different things, and the model treats them as one.

What it costs you. CourseID is the key of Course, so it can only hold one Semester and one Year at a time. The day CS301 runs in both a spring and an autumn term, you either overwrite one offering or invent a second Course row with a fake key. Either way you lose the ability to see all past offerings of the course.

The fix. Pull Semester and Year off Course. Add a weak entity Section, identified by Course plus Year, Semester and a section number, and hang the term details there instead.

Issue 2: ENROLLS connects Student to Course, not to a Section

The flaw. Students register for one specific section with its own instructor and timing, but the model only lets a student attach to a course as a whole.

What it costs you. You cannot tell which section a student is actually in, so you cannot derive their instructor, room, or timetable. Worse, a student who repeats a failed course cannot be represented twice, because the Student to Course pair can only exist once. Grade history for repeat attempts disappears.

The fix. Move ENROLLS so it links Student to Section instead of Course, and put RegistrationDate and Grade on that relationship.

Issue 3: TEACHES connects Instructor to Course, not to a Section

The flaw. Different sections of the same course can have different instructors, but TEACHES is drawn at the course level.

What it costs you. If Section A of a course is taught by one instructor and Section B by another, the model has no way to record both. Teaching load per instructor per term becomes impossible to calculate, and students cannot find out who actually teaches their section.

The fix. Re-anchor TEACHES to Section.

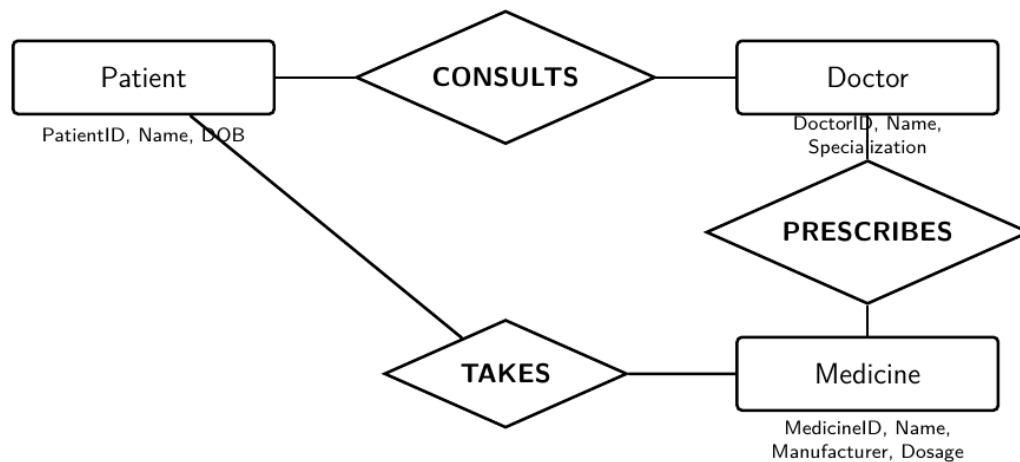
Issue 4: HELD_IN connects Classroom to Course, not to a Section

The flaw. Room allocation can differ between sections running in the same term, but the relationship is drawn at course level.

What it costs you. Two sections meeting at the same time in different rooms cannot both be recorded. Room clash detection, which is really the whole point of tracking rooms, becomes impossible, since clashes happen between sections, not between courses.

The fix. Re-anchor HELD_IN to Section as well.

Question 2: Hospital Prescription System



As given in the assignment (page 5).

As given

- Entities: Patient (PatientID, Name, DOB), Doctor (DoctorID, Name, Specialization), Medicine (MedicineID, Name, Manufacturer, Dosage).
- Relationships: Patient CONSULTS Doctor, Doctor PRESCRIBES Medicine, Patient TAKES Medicine.
- PRESCRIBES and TAKES are two separate relationships that both connect to Medicine, with no direct link tying a doctor's prescription to the patient it was written for.

Issue 1: CONSULTS is a bare relationship

The flaw. A consultation is a real event with its own date, time and clinical notes, but the model treats it as a plain fact that a patient and a doctor are associated.

What it costs you. The relationship can hold the pair (patient, doctor) only once. A patient who sees the same doctor every month for two years looks identical to one who saw them a single time. You lose the visit date, the diagnosis recorded that day, and any way to say which visit a prescription actually came from.

The fix. Turn Consultation into its own weak entity, identified by patient, doctor and the date and time of the visit.

Issue 2: PRESCRIBES and TAKES split one fact into two

The flaw. 'Doctor prescribed medicine to patient on this date' is a single four way fact, but the diagram breaks it into Doctor to Medicine and Patient to Medicine as two separate, unrelated links.

What it costs you. If two doctors prescribe the same drug to two different patients, joining the two relationships back together gives you every possible combination, with no way to know which doctor actually prescribed to which patient. The date is missing entirely too, so you cannot even say when.

The fix. Delete both relationships. Build a Prescription entity off Consultation, since Consultation already fixes who, whom and when, and link Prescription to Medicine through a prescription line.

Issue 3: Dosage sits on Medicine

The flaw. Dosage depends on the patient and the visit, but it is stored as a fixed property of the drug itself.

What it costs you. Every patient given that medicine is forced to share one dosage value. A child needing a smaller dose and an adult needing a larger one cannot both be represented for the same drug, and a dose that changes across visits for the same patient is impossible to record.

The fix. Remove Dosage from Medicine. Put it on the individual prescription line, where it actually belongs.

Issue 4: No line level detail, so a drug cannot be re-prescribed differently

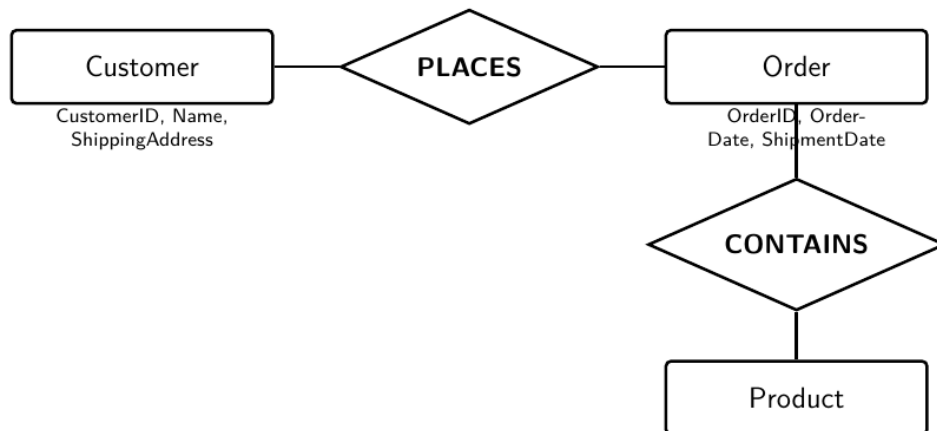
The flaw. Nothing in the model carries frequency, duration or instructions, and there is no way for the same medicine to appear twice for the same patient with different details.

What it costs you. A five day course of an antibiotic in January and a seven day course of the same antibiotic in June are clinically different events, but the model can only hold one Patient to Medicine fact and collapses them into one. Dispensing instructions and course length are lost entirely.

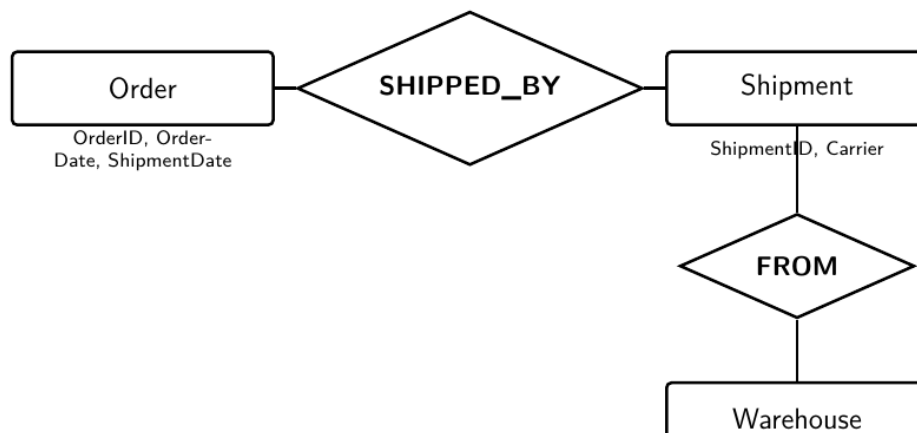
The fix. Give each prescription a set of prescription lines, one per medicine, carrying Dosage, Frequency, Duration and Instructions. Because each line belongs to a specific Prescription tied to a specific Consultation, repeat prescriptions are naturally distinct.

Question 3: E-Commerce Order Fulfilment

Flawed ER Design — Part A



Flawed ER Design — Part B



As given in the assignment, Part A and Part B (pages 7 and 8).

As given

- Part A entities: Customer (CustomerID, Name, ShippingAddress), Order (OrderID, OrderDate, ShipmentDate), Product (ProductID, Name, Price, Quantity).
- Part A relationships: Customer PLACES Order, Order CONTAINS Product.
- Part B entities: Order, Shipment (ShipmentID, Carrier), Warehouse (WarehouseID, City).
- Part B relationships: Order SHIPPED_BY Shipment, Shipment FROM Warehouse.
- Quantity is shown as an attribute of Product, and ShipmentDate as a single attribute of Order.

Issue 1: ShippingAddress sits on Customer

The flaw. Every order can ship to a different address, but the model gives the customer only one address, permanently.

What it costs you. You cannot ship one order home and another to an office. Worse, if the customer later updates their saved address, every past order silently points to the new address too, since the order never recorded its own. Past delivery records become wrong retroactively.

The fix. Move the address onto Order itself, captured at the moment the order is placed, so it stays fixed even if the customer's profile changes later.

Issue 2: Quantity sits on Product, and CONTAINS has no real substance

The flaw. How many units were ordered is a fact about a specific order line, not a fact about the product.

What it costs you. A product can only hold one Quantity value system wide, so ordering three units in one order and five in another cannot both exist. Line totals, order totals and stock deduction all become impossible to compute, since there is nowhere to store the actual amount ordered.

The fix. Turn CONTAINS into a real OrderLine entity carrying Quantity and the price at the time of order. Remove Quantity from Product entirely.

Issue 3: ShipmentDate sits on Order

The flaw. An order can be split into several shipments, each dispatched on its own date, but the model gives the order a single shipment date.

What it costs you. Only one dispatch date can be stored per order, so a split shipment forces you to either overwrite the first date or ignore the rest. Per shipment tracking and delivery estimates for individual parcels become impossible.

The fix. Drop ShipmentDate from Order and put a proper DispatchDate on Shipment, right alongside Carrier where it already lives.

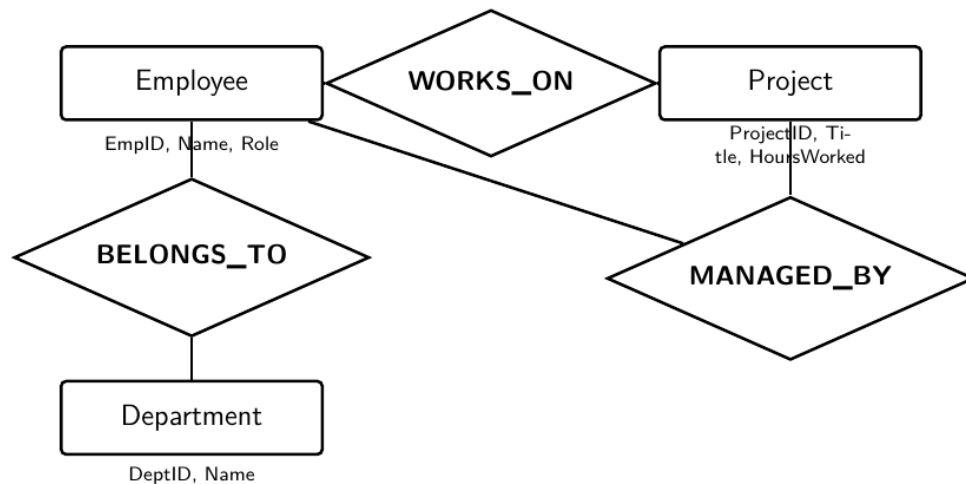
Issue 4: SHIPPED_BY only connects whole orders to whole shipments

The flaw. An order line can be split and partially shipped from different warehouses, but the relationship only says that a shipment belongs to an order, with no detail on which products or how many units.

What it costs you. You cannot tell whether an order is fully or partially fulfilled, cannot compute how much of a line is still outstanding, and cannot represent six units coming from one warehouse and four from another for the same line.

The fix. Add a ShipmentLine entity between Shipment and OrderLine that records the quantity shipped on each line. Comparing that to the ordered quantity tells you exactly what is fulfilled and what is still owed.

Question 4: Project Staffing and Roles



As given in the assignment (page 10).

As given

- Entities: Employee (EmpID, Name, Role), Project (ProjectID, Title, HoursWorked), Department (DeptID, Name).
- Relationships: Employee WORKS_ON Project, Employee BELONGS_TO Department, Project MANAGED_BY Employee.
- Role is shown as an attribute of Employee, and HoursWorked as a single attribute of Project.

Issue 1: Role sits on Employee

The flaw. An employee's role can differ from one project to the next, but Role is stored as a single fixed property of the employee.

What it costs you. An employee can only hold one role at a time, so someone who leads on one project and contributes as a regular developer on another cannot have both roles recorded. Any query about who is leading which project becomes unreliable.

The fix. Remove Role from Employee and place it on the assignment record instead, so it can vary per project.

Issue 2: WORKS_ON cannot hold repeated stints

The flaw. An employee can join a project, leave it, and rejoin it later, but WORKS_ON is a plain relationship that can only hold one instance of Employee paired with Project.

What it costs you. A second stint on the same project cannot be recorded separately from the first, so the gap in between, the exact dates of each stint, and the true history of who was on the team at any given time all disappear.

The fix. Turn WORKS_ON into an Assignment entity identified by employee, project and start date. Because the start date is part of the identity, a second stint is automatically a separate record rather than a clash.

Issue 3: HoursWorked sits on Project

The flaw. Hours need to be tracked per employee, per project, per time period, but the model stores one single number on the project as a whole.

What it costs you. You get one total with no way to know who contributed it or when. Payroll allocation, billing by person, and monthly burn rate reporting are all impossible with just one aggregate number sitting on the project.

The fix. Log hours per assignment and per period underneath it. Treat any project level total as something you calculate afterward, not something you store directly.

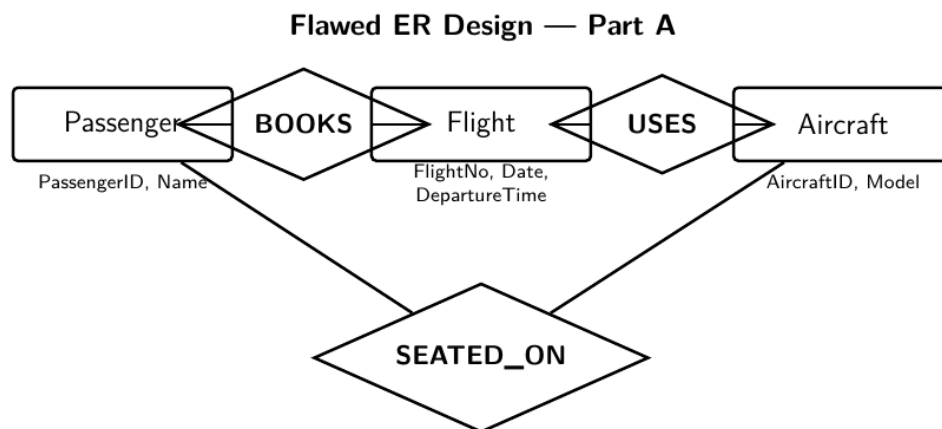
Issue 4: MANAGED_BY only remembers the current manager

The flaw. Project managers can change over the life of a project, and management history has to be kept for audit purposes, but the relationship only ever holds the current one.

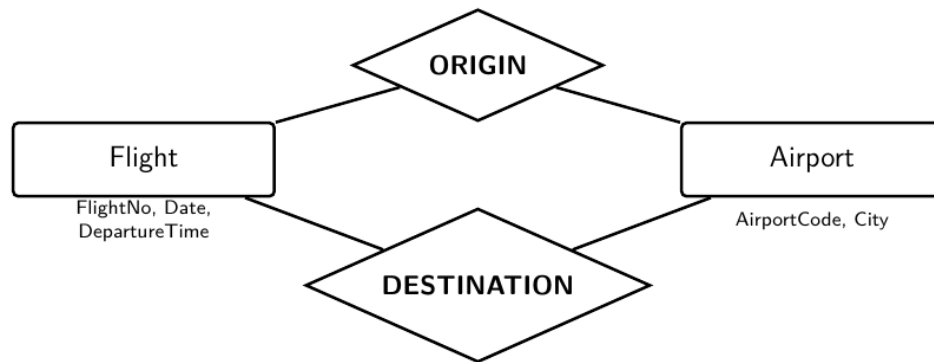
What it costs you. The moment a new manager takes over, the old relationship instance is overwritten and the previous manager disappears from the record. Anything that needs to know who was accountable at a past point in time cannot be answered.

The fix. Reify management into a ManagementPeriod entity with a start date and an end date. The current manager is simply the one whose end date is still open, and everyone before them stays on record.

Question 5: Airline Booking and Seat Assignment



Flawed ER Design — Part B



As given in the assignment, Part A and Part B (pages 12 and 13).

As given

- Part A entities: Passenger (PassengerID, Name), Flight (FlightNo, Date, DepartureTime), Aircraft (AircraftID, Model).
- Part A relationships: Passenger BOOKS Flight, Flight USES Aircraft, and SEATED_ON connecting Passenger and Aircraft.
- Part B entities: Flight, Airport (AirportCode, City).
- Part B relationships: Flight ORIGIN Airport, Flight DESTINATION Airport.
- Date is shown as a single attribute of Flight itself, and SEATED_ON carries no seat number attribute.

Issue 1: Flight mixes up the schedule with a specific day's flight

The flaw. A flight number like AI-101 runs on many different dates, but the model stores Date as a plain attribute of Flight itself.

What it costs you. If FlightNo alone identifies Flight, the flight can only exist on one date ever, which directly breaks the rule that it operates on many dates. If you instead key on FlightNo plus Date, the departure time and route end up repeated for every single date, and editing one day's route can accidentally change all of them.

The fix. Split Flight into two levels: Flight holds the fixed schedule, and a new weak entity FlightInstance, keyed by flight number and date, holds everything specific to that day's departure.

Issue 2: BOOKS has no concept of a PNR or an itinerary

The flaw. A real booking covers a reference number, one or more passengers, and one or more flight segments, but BOOKS is just a plain link between one passenger and one flight.

What it costs you. A family of four traveling together cannot be grouped under one reservation, a journey with a layover cannot be represented as a connected itinerary, and a passenger cannot even be rebooked on the same flight after a cancellation, since the pair can only exist once.

The fix. Add a Booking entity holding the PNR, and a Segment entity underneath it that ties together the booking, the passenger and the specific flight instance for each leg of the journey.

Issue 3: USES connects Aircraft to Flight, not to the day's flight

The flaw. The aircraft flying a route can change on a specific day due to maintenance or availability, but the relationship is drawn at the flight number level, which spans every date.

What it costs you. Since every date of a flight number is forced to share the same aircraft, a one day tail swap cannot be recorded without rewriting the aircraft for every other date too, including flights that already flew. Seat maps and capacity limits for a given day end up wrong.

The fix. Move USES down to connect Aircraft to FlightInstance, so equipment can change from one day to the next without touching any other date.

Issue 4: SEATED_ON connects Passenger to Aircraft, and no seat number exists anywhere

The flaw. A seat assignment really depends on the passenger, the specific flight instance, and the seat number itself, but the model links a passenger straight to an aircraft with no seat attribute at all.

What it costs you. As drawn, a passenger is permanently seated on a physical airframe regardless of date, which does not mean anything useful. You cannot build a seat map for a given departure, cannot stop the same seat being given to two people on the same flight, and cannot give a passenger a different seat on their outbound versus their return leg.

The fix. Drop SEATED_ON. Put SeatNo directly on Segment, since Segment already ties together the passenger and the specific flight instance, and enforce that no two segments on the same flight instance can share a seat number.